

CodeQL workshop



Agenda

1. Identifying vulnerabilities in code
2. Writing queries with CodeQL
3. Workshop: <https://gh.io/sosf-codeql>

Who am I

Sylwia Budzynska

- Security researcher
- 80+ CVEs, mostly web-related vulns
- Currently in rotation working on private vulnerability reporting feature

How do we find vulnerabilities?

Let's start by examining an example vulnerability and how it could be detected.

Example vuln

CVE-2024-32022, CVE-2024-32026, CVE-2024-32025, CVE-2024-32027

https://securitylab.github.com/advisories/GHSL-2024-019_GHSL-2024-024_kohya_ss/

```
import gradio as gr
import os

def execute_cmd(folder):
    cmd = f"python caption.py --dir={folder}"
    os.system(cmd)

with gr.Blocks() as demo:
    gr.Markdown("Create caption files for images")
    with gr.Row():
        folder = gr.Textbox(label="Directory to caption")
        logs = gr.Checkbox(label="Add verbose logs")

    btn = gr.Button("Run")
    btn.click(fn=execute_cmd, inputs=[folder])

if __name__ == "__main__":
    demo.launch(debug=True)
```

Example vuln

CVE-2024-32022, CVE-2024-32026, CVE-2024-32025, CVE-2024-32027

https://securitylab.github.com/advisories/GHSL-2024-019_GHSL-2024-024_kohya_ss/

```
import gradio as gr
import os

def execute_cmd(folder):
    cmd = f"python caption.py --dir={folder}"
    os.system(cmd)

with gr.Blocks() as demo:
    gr.Markdown("Create caption files for images")
    with gr.Row():
        folder = gr.Textbox(label="Directory to caption")
        logs = gr.Checkbox(label="Add verbose logs")

    btn = gr.Button("Run")
    btn.click(fn=execute_cmd, inputs=[folder])

if __name__ == "__main__":
    demo.launch(debug=True)
```

Example vuln

CVE-2024-32022, CVE-2024-32026, CVE-2024-32025, CVE-2024-32027

https://securitylab.github.com/advisories/GHSL-2024-019_GHSL-2024-024_kohya_ss/

```
import gradio as gr
import os

def execute_cmd(folder):
    cmd = f"python caption.py --dir={folder}"
    os.system(cmd)

with gr.Blocks() as demo:
    gr.Markdown("Create caption files for images")
    with gr.Row():
        folder = gr.Textbox(label="Directory to caption")
        logs = gr.Checkbox(label="Add verbose logs")

    btn = gr.Button("Run")
    btn.click(fn=execute_cmd, inputs=[folder])

if __name__ == "__main__":
    demo.launch(debug=True)
```

Identifying the vuln

This command injection is caused by the user-controlled folder (**source**) used directly in a command, in the `os.system` call (**sink**).

Example PoC payload:

```
foo || touch file.txt
```

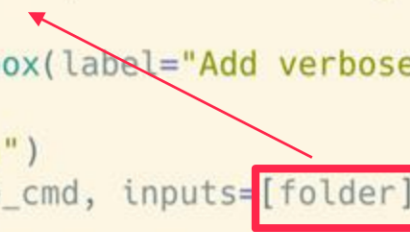
```
import gradio as gr
import os

def execute_cmd(folder):
    cmd = f"python caption.py --dir={folder}"
    os.system(cmd)

with gr.Blocks() as demo:
    gr.Markdown("Create caption files for images")
    with gr.Row():
        folder = gr.Textbox(label="Directory to caption")
        logs = gr.Checkbox(label="Add verbose logs")

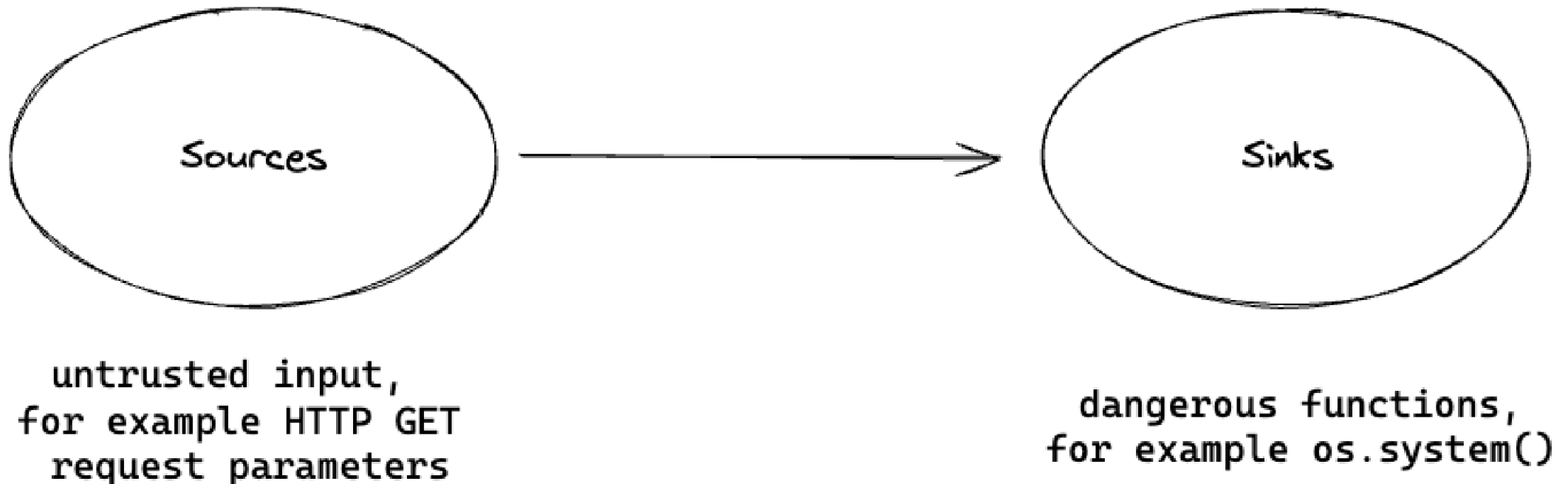
    btn = gr.Button("Run")
    btn.click(fn=execute_cmd, inputs=[folder])

if __name__ == "__main__":
    demo.launch(debug=True)
```

A red arrow points from the `folder` argument in the `execute_cmd` function call (`execute_cmd(folder)`) to the `folder` parameter in the function definition (`def execute_cmd(folder):`). Another red arrow points from the `folder` variable in the `execute_cmd` function to the `cmd` string assignment (`cmd = f"python caption.py --dir={folder}"`). The `cmd` variable is highlighted with a red box, and the `folder` variable in the `inputs` list of the `click` method is also highlighted with a red box.

Sources and sinks

For a vulnerability to be present, there has to be a code path between the **source** and the **sink**—a **data flow**.

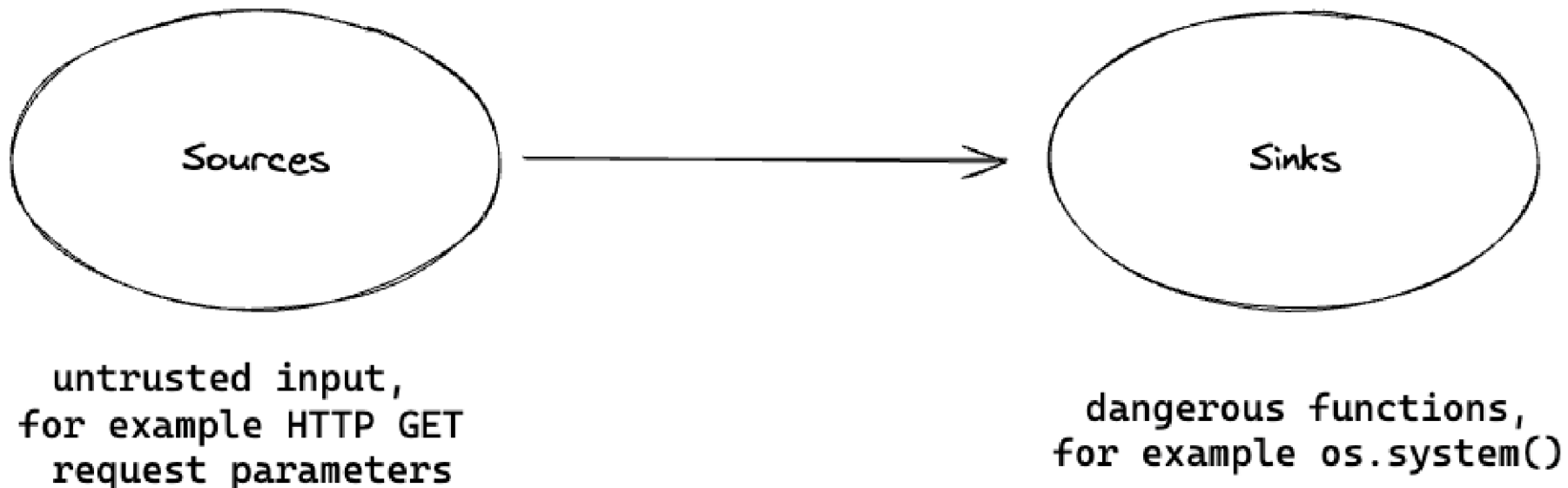


Problem

An application often has hundreds of sources and hundreds of sinks.

Checking manually, if any of the sources connect to the sinks (or vice versa, any of the sinks to the sources), takes time.

Is it possible to automate finding sources, sinks and data flow paths between them?



Yes,

we can check for data flows
between sources and sinks by
using

CodeQL



```
import python
import semantics.python.ApiGraphs

from API.allNode node
where node =
API:moduleImport("django").getMember("db").getMember("connection").getMember("cursor")
.getReturn().getReturn().getMember("execute").getACall()
and
node.getFile().getRelativePath().regexMatch("2/challenge-1/.+")
select node
```

**Q: Have you ever used
CodeQL before?**

Code Scanning - CodeQL in your CI

- finds vulnerabilities before they are merged into the codebase with automated CodeQL scans
- runs CodeQL using Actions runners
- extensible, with support for other SAST tools
- free for public repositories on GitHub

Code scanning alerts / #40

Uncontrolled command line

 Open in `main` 2 days ago

 Speed up the remediation of this alert with [Copilot Autofix for CodeQL](#)

 Generate fix

4/vulnerable-code-snippets/cmd-list.py:6 

```
3
4 def execute_cmd(folder, logs):
5     cmd = f"python caption.py --dir={folder} --logs={logs}"
6     os.system(cmd)
```

This command line depends on a [user-provided value](#).
This command line depends on a [user-provided value](#).

CodeQL [Show paths](#)

```
7     return f"Command: {cmd}"
8
9
```

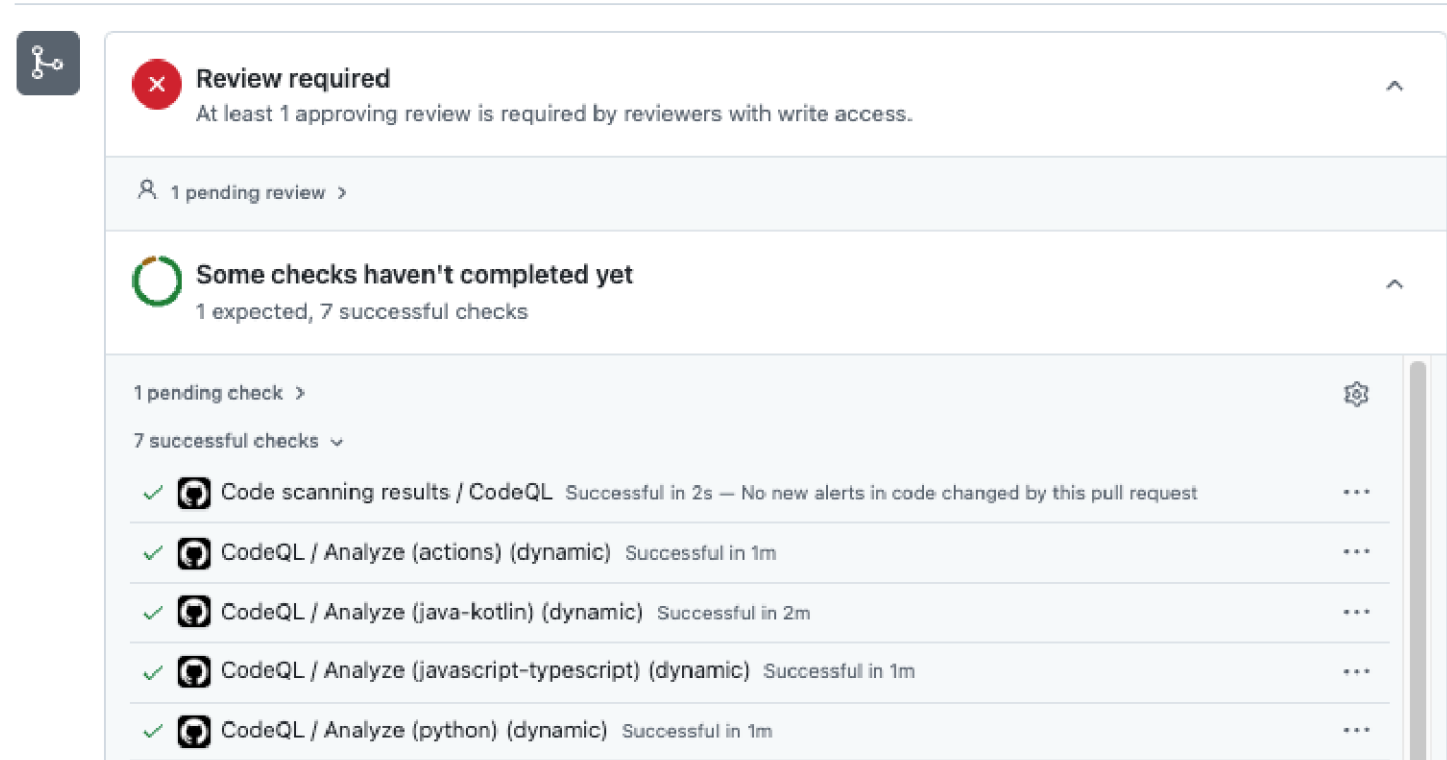
Tool	Rule ID	Query
CodeQL	py/command-line-injection	View source

Code that passes user input directly to `exec`, `eval`, or some other library routine that executes a command, allows the user to execute malicious code.

[Show more](#) ▾

Supported languages

C/C++, C#, GitHub
Actions, Go, Java, Kotlin,
Javascript, Python, Ruby,
Rust, Swift, Typescript



The screenshot displays the GitHub Actions status bar for a pull request. It features a red 'x' icon for 'Review required' and a green circle icon for 'Some checks haven't completed yet'. The 'Review required' section indicates that at least one approving review is needed from reviewers with write access. The 'Some checks haven't completed yet' section shows that 1 check is expected and 7 checks are successful. Below this, a list of checks is shown, including 'Code scanning results / CodeQL' and several 'CodeQL / Analyze' checks for different languages (actions, java-kotlin, javascript-typescript, python), all of which are successful.

 **Review required** 
At least 1 approving review is required by reviewers with write access.

 1 pending review >

 **Some checks haven't completed yet** 
1 expected, 7 successful checks

1 pending check > 

7 successful checks v

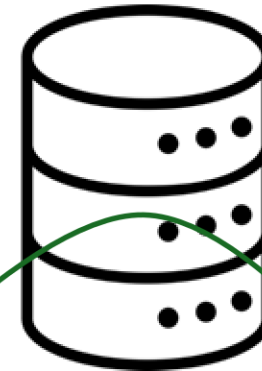
- ✓  Code scanning results / CodeQL Successful in 2s — No new alerts in code changed by this pull request ...
- ✓  CodeQL / Analyze (actions) (dynamic) Successful in 1m ...
- ✓  CodeQL / Analyze (java-kotlin) (dynamic) Successful in 2m ...
- ✓  CodeQL / Analyze (javascript-typescript) (dynamic) Successful in 1m ...
- ✓  CodeQL / Analyze (python) (dynamic) Successful in 1m ...

How CodeQL works

Source code

```
503         message =  
504         if not hasattr(self, '_headers_buffer'):  
505             self._headers_buffer = []  
506             self._headers_buffer.append(("Xs %d %s\r\n" %  
507                 (self.protocol_version, code, message)).encode(  
508                     'latin-1', 'strict'))  
509  
510     def send_header(self, keyword, value):  
511         """Send a MIME header to the headers buffer."""  
512         if self.request_version != 'HTTP/0.9':  
513             if not hasattr(self, '_headers_buffer'):  
514                 self._headers_buffer = []  
515                 self._headers_buffer.append(  
516                     ("%s: %s\r\n" % (keyword, value)).encode('latin-1', 'strict'))  
517  
518             if keyword.lower() == 'connection':  
519                 if value.lower() == 'close':  
520                     self.close_connection = True  
521                 elif value.lower() == 'keep-alive':  
522                     self.close_connection = False  
523
```

CodeQL database



CodeQL queries

Vulnzzz

CodeQL detects vulnerabilities in two steps:

1. Parses your code into a relational **database** of facts, which contains information about **elements of our source code**, f.ex. classes and functions and relationships between these elements
2. Runs CodeQL queries on that database, which detect **vulnerable patterns** in your code, e.g. SQL injection, command injection

<https://github.blog/2023-03-31-codeql-zero-to-hero-part-1-the-fundamentals-of-static-analysis-for-vulnerability-research/>

How CodeQL works pt. 2

```
Module(  
  body=[  
    ImportFrom(  
      lineno=2,  
      col_offset=0,  
      end_lineno=2,  
      end_col_offset=32,  
      module='django.db',  
      names=[alias(lineno=2, col_offset=22, end_lineno=  
end_col_offset=32, name='connection', asname=None)],  
      level=0,  
    ),  
    FunctionDef(  
      lineno=5,  
      col_offset=0,  
      end_lineno=7,  
      end_col_offset=78,  
      name='show_user',  
      args=arguments(  
        posonlyargs=[],  
        args=[  
          arg(lineno=5, col_offset=14, end_lineno=5  
end_col_offset=21, arg='request', annotation=None,  
type_comment=None),  
          arg(lineno=5, col_offset=23, end_lineno=5  
end_col_offset=31, arg='username', annotation=None,  
type_comment=None),  
        ],  
        vararg=None,  
        kwonlyargs=[],
```

```
. from django.db import connection  
.   
. def show_user(request, username):  
.     with connection.cursor() as cursor:  
.         cursor.execute("SELECT * FROM users WHERE username = '%s'" % username)
```



What does a CodeQL
query look like?

Import: lets us reuse logic
defined in other libraries

```
import python
```

```
from <type> <name>           //variables used in the query  
  
where <conditions for variables>  
  
select <output>              //results, referring to the variables
```

Query clause: describes what we are trying to find

Import: lets us reuse logic
defined in other libraries

```
import python  
import semmle.python.ApiGraphs
```

```
from API::CallNode call  
where call = API::moduleImport("os")  
            .getMember("system")  
            .getACall()  
select call, "Call to os.system"
```

Query clause: describes what we are trying to find



Import: lets us reuse logic defined in other libraries

```
import python
import semmle.python.ApiGraphs
```

```
from API::CallNode call
where call = API::moduleImport("os")
            .getMember("system")
            .getACall()
select call, "Call to os.system"
```

Query clause: describes what we are trying to find

#select ▾

35 results

#	[0]	[1]
1	ControlFlowNode for cmd	First argument of an `os.system` call
2	ControlFlowNode for cmd	First argument of an `os.system` call
3	ControlFlowNode for BinaryExpr	First argument of an `os.system` call
4	ControlFlowNode for run_cmd	First argument of an `os.system` call
5	ControlFlowNode for run_cmd	First argument of an `os.system` call
6	ControlFlowNode for run_cmd	First argument of an `os.system` call
7	ControlFlowNode for run_cmd	First argument of an `os.system` call
8	ControlFlowNode for run_cmd	First argument of an `os.system` call
9	ControlFlowNode for run_cmd	First argument of an `os.system` call
10	ControlFlowNode for run_cmd	First argument of an `os.system` call
11	ControlFlowNode for run_cmd	First argument of an `os.system` call
12	ControlFlowNode for run_cmd	First argument of an `os.system` call
13	ControlFlowNode for run_cmd	First argument of an `os.system` call
14	ControlFlowNode for run_cmd	First argument of an `os.system` call
15	ControlFlowNode for run_cmd	First argument of an `os.system` call



Building blocks of a query

Predicates

Like functions, but better!

Create reusable logic and give it a name.

Mini from-where-select

Just a query

```
from API::CallNode call
where
    call = API::moduleImport("os")
        .getMember("system").getACall()
select call, "Call to os.system"
```

Using a predicate

```
predicate isOsSystemSink(API::CallNode
call) {
    call = API::moduleImport("os")
        .getMember("system").getACall()
}
```

```
from API::CallNode call
where isOsSystemSink(call)
select call, "Call to os.system"
```

Classes

Describe sets of values.

Using a predicate

```
predicate isOsSystemSink(API::CallNode  
call) {  
    call = API::moduleImport("os")  
    .getMember("system").getACall()  
}
```

```
from API::CallNode call  
where isOsSystemSink(call)  
select call, "Call to os.system"
```

Using a class

```
class OsSystemSink extends API::CallNode  
{  
    OsSystemSink() {  
        this = API::moduleImport("os")  
        .getMember("system").getACall()  
    }  
}
```

```
from API::CallNode call  
where call instanceof OsSystemSink  
select call, "Call to os.system"
```

Using a predicate

```
predicate isOsSystemSink(API::CallNode  
call) {  
    call = API::moduleImport("os")  
    .getMember("system").getACall()  
}
```

```
from API::CallNode call  
where isOsSystemSink(call)  
select call, "Call to os.system"
```

Using a class pt. 2

```
class OsSystemSink extends API::CallNode  
{  
    OsSystemSink() {  
        this = API::moduleImport("os")  
        .getMember("system").getACall()  
    }  
}
```

```
from OsSystemSink s  
select s, "Call to os.system"
```

Source and sink models in CodeQL

CodeQL has hundreds of types representing sources, sinks, etc. and CodeQL checks for a data flow path between these predefined sources and sinks.

When we find a new source or a sink and write a query for them, we say that we **model** sources and sinks in CodeQL.

For example, all sources are modelled as **RemoteFlowSource**, which we can query.

```
import gradio as gr
import os

def execute_cmd(folder):
    cmd = f"python caption.py --dir={folder}"
    os.system(cmd)

with gr.Blocks() as demo:
    gr.Markdown("Create caption files for images")
    with gr.Row():
        folder = gr.Textbox(label="Directory to caption")
        logs = gr.Checkbox(label="Add verbose logs")

    btn = gr.Button("Run")
    btn.click(fn=execute_cmd, inputs=[folder])

if __name__ == "__main__":
    demo.launch(debug=True)
```


Multi-repository variant analysis (MRVA)

Run your CodeQL query against hundreds of repositories.

▼ VARIANT ANALYSIS REPOSITORIES

 Top 10 repositories

 Top 100 repositories

 Top 1000 repositories

View Autofixes

Copy repos

ql

</>View query

RESULTS

7

REPOSITORIES

997/999

DURATION

-

RUNNING

⌵

View actions logs

Filter by repository owner/name

All

Number of

Alerts

☐

>

4

public

✓

☆ 602

☐

>

2

public

✓

☆ 8.4k

☐

>

1

public

✓

☆ 15k

☐

>

0

public

✓

☆ 7.4k

☐

>

0

public

✓

☆ 81k

☐

>

0

public

✓

☆ 12k

☐

>

0

public

✓

☆ 2.4k

☐

>

0

public

✓

☆ 2.7k

☐

>

0

public

✓

☆ 8.9k

☐

>

0

public

✓

☆ 1.1k

☐

>

0

public

✓

☆ 5.5k

☐

>

0

public

✓

☆ 4.8k

☐

>

0

public

✓

☆ 3.2k

☐

>

0

public

✓

☆ 3.6k

☐

>

0

public

✓

☆ 56k

☐

>

0

public

✓

☆ 34k

Q: What could be considered a source or a sink in your project?

- For web applications:
 - CodeQL will find most sources and sinks
- For libraries:
 - What endpoints are we exposing to the developers - could they be considered a source or a sink
 - What potentially dangerous functionality do we have internally in our code

Links

CodeQL workshop:

- Short URL: gh.io/sosf-codeql
- <https://github.com/sylwia-budzynska/codeql-workshop>

CodeQL docs

- <https://codeql.github.com/docs/>

CodeQL zero to hero blog post series

- <https://github.blog/author/sylwiabudzynska/>

GitHub Security Lab

- <https://securitylab.github.com/>